Node.js — Day 18 Complete Notes

Topic: Promises & Async/Await
Goal: Understand Promises, resolve/reject, then/catch, async, await, and try/catch.

1. What is a Promise?

A **Promise** represents an operation whose result will be available later. It is commonly used for asynchronous work in JavaScript and Node.js.

Analogy: Think of ordering food. Your order is initially waiting, then it either arrives successfully or something goes wrong.

2. Promise States

State	Meaning
pending	The operation is still working.
fulfilled	The operation completed successfully.
rejected	The operation failed.

3. Creating a Promise

```
const promise = new Promise((resolve, reject) => {
  resolve("Success!");
});
```

resolve() means success. **reject()** means failure.

4. Handling Promise Results

```
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  });
```

.then() handles success; **.catch()** handles failure.

5. async Functions

An **async function always returns a Promise**, even when it returns an ordinary value.

```
async function getData() {
  return "Data received";
}

getData().then((result) => {
  console.log(result);
});
```

6. await

await lets us wait for a Promise's result and use that result directly. It is normally used inside an **async function**.

```
async function getData() {
  const result = await promise;
  console.log(result);
}
```

Mental model: await promise → wait for the Promise's result → continue with the result.

7. Error Handling with try/catch

```
async function getData() {
  try {
    const result = await promise;
    console.log(result);
  } catch (error) {
    console.log(error);
  }
}
```

If the Promise fails while using await, the **catch** block handles the error.

8. Practical Node.js Example

```
function getUser() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({
        name: "Bhavana",
        age: 26
      });
    }, 1000);
  });
}

async function showUser() {
  try {
    const user = await getUser();
    console.log(user.name);
    console.log(user.age);
  } catch (error) {
    console.log(error);
  }
}

showUser();
```

9. Callback → Promise → async/await

Callbacks run a function later when async work is ready. Promises represent a future success/failure result. async/await provides a cleaner way to work with Promises.

10. Mini Challenge — Completed

```
async function start() {
  const message = await getData();
  console.log(message);
}

start();
```

The answer used **message** for both blanks: const message = await getData() and console.log(message).

11. Important Rules

- Promise → represents a future result.
- resolve() → success.
- reject() → failure.
- .then() → handles success.
- .catch() → handles failure.
- async → makes a function return a Promise.
- await → waits for a Promise's result.
- try/catch → handles errors with async/await.
- await is normally used inside an async function.

12. Day 18 Quick Revision

Promise: pending → fulfilled/rejected

Promise style: resolve() → .then() reject() → .catch()

async/await style: async function → await Promise → try/catch

■ Key Takeaway

Promises provide a structured way to work with asynchronous results. **async/await** provides a cleaner and easier-to-read way to consume those Promises. This pattern will be important for APIs, databases, files, and other backend operations.